

UNDECIDABILITY

Dr. A. Beulah
AP/CSE

LEARNING OBJECTIVE

- To Design Turing machines for any Languages (K3)
 - To Understand the concept of Universal Turing Machine

NEED FOR A UNIVERSAL TM

- Each TM appears to be specialized at solving one particular problem. (Hardwired)
- Computers solve many problems → General purpose computers (Re-programmable)
- It is possible to invent a single TM which can be used to compute any computable sequence.

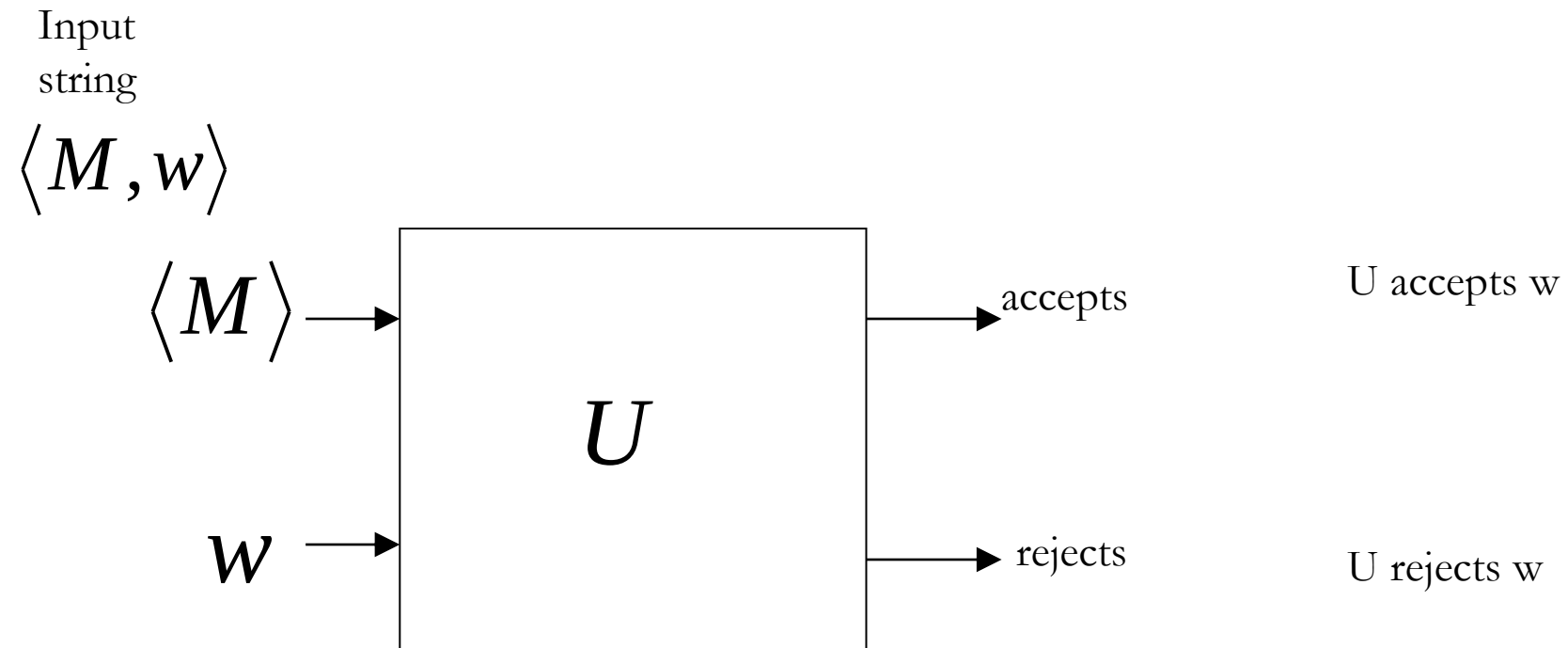
UNIVERSAL TM

- A **universal Turing machine (UTM)** is a Turing machine that can simulate an arbitrary Turing machine on arbitrary input.
- The universal machine essentially achieves this by reading both the description of the machine to be simulated as well as the input thereof from its own tape.
- The UTM played an important early role in stimulating the development of stored-program computers.
- A universal TM can execute any algorithm, provided it receives an input string that describes the algorithm and any data it is to process.

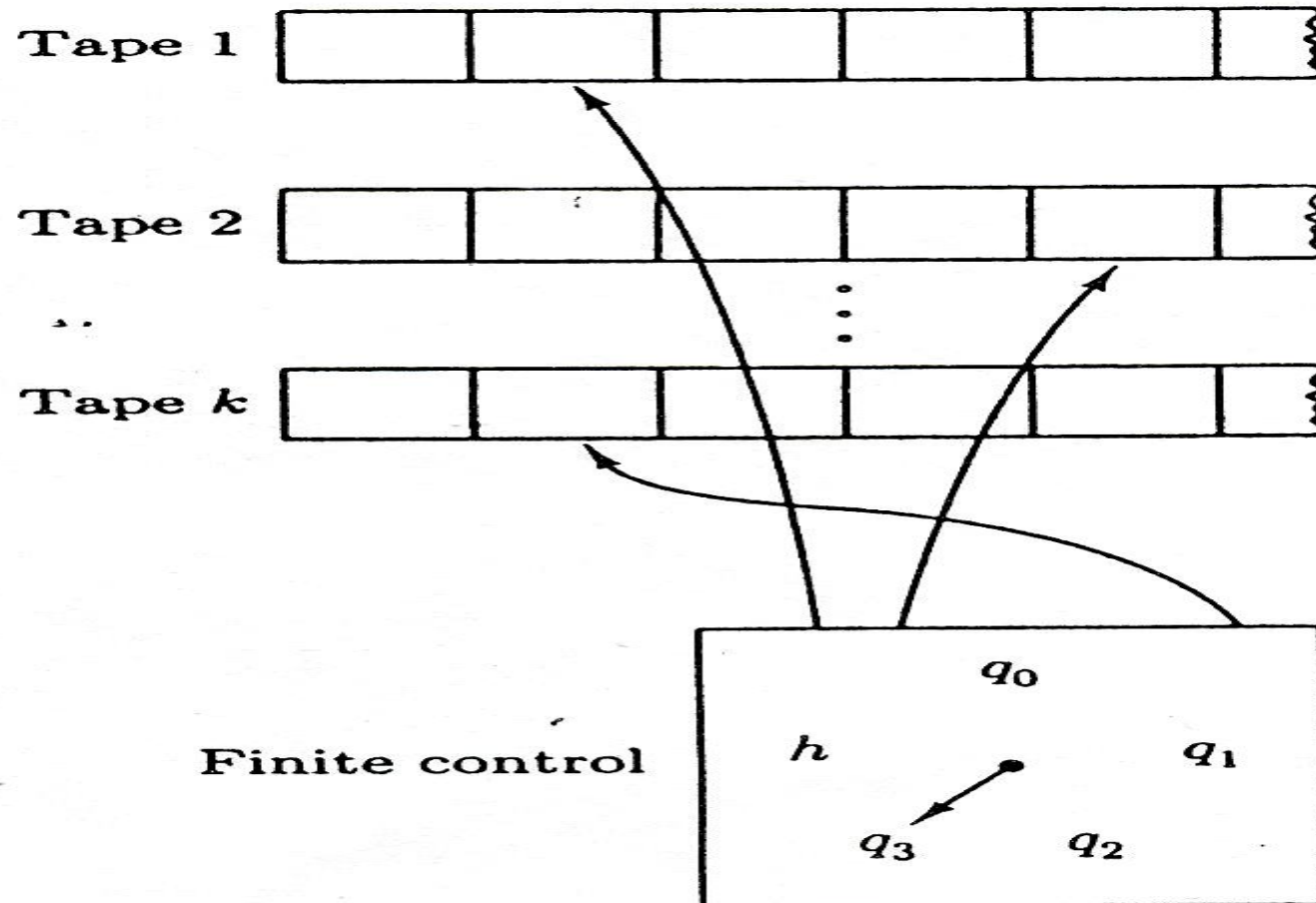
DEFINITION

- A *UTM* is a Turing machine U that works as follows.
 - It is assumed to receive an input string of the form $e(M)e(w)$, where M is an arbitrary TM, w is a string over the input alphabet of M , and e is an encoding function whose values are strings in $\{0, 1\}^*$. The computation performed by U on this input string satisfies these two properties:
 1. U accepts the string $e(M)e(w)$ if and only if M accepts w .
 2. If M accepts w and produces output y , then U produces output $e(y)$.

DEFINITION UTM



MULTIPLE TAPE TURING MACHINE



ORGANIZATION OF UTM

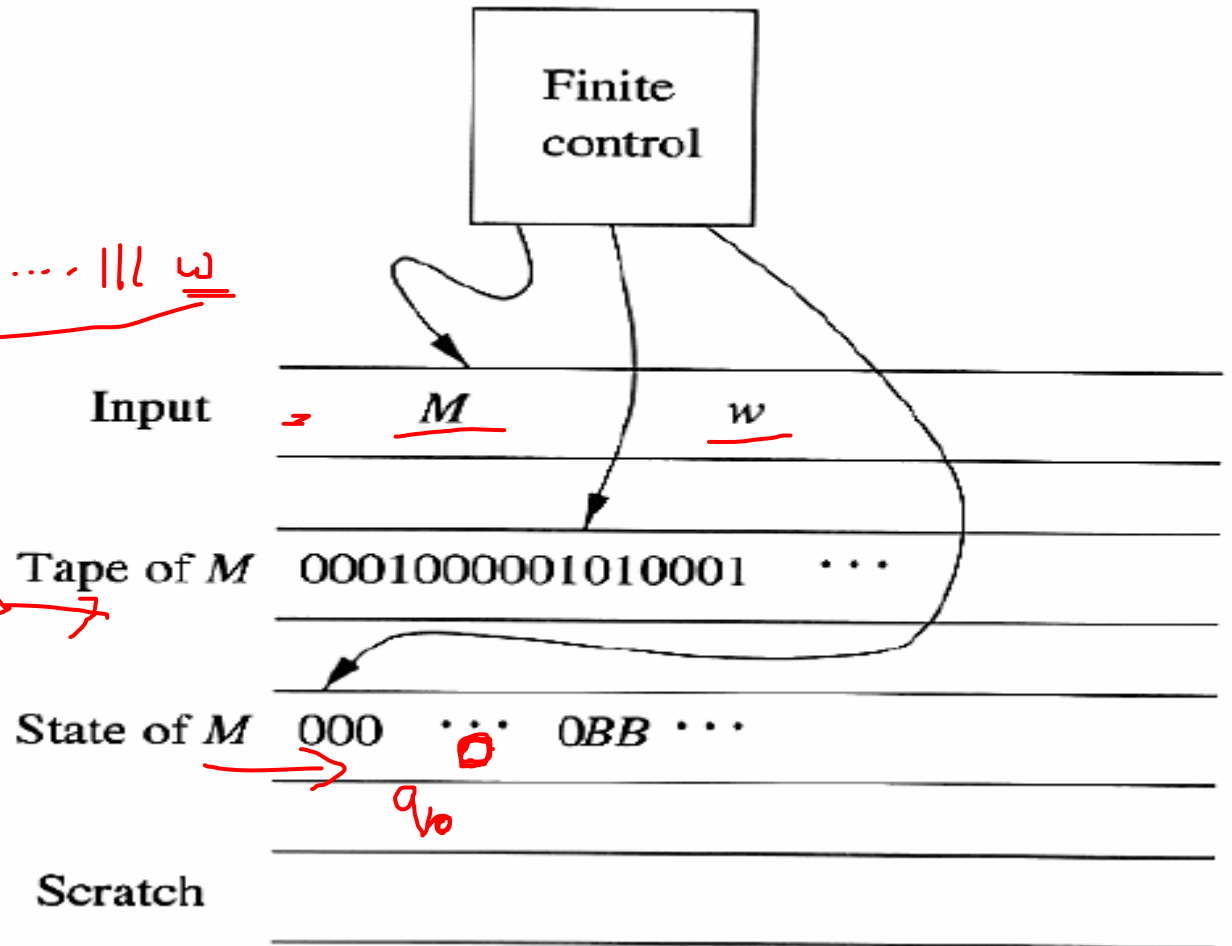
$$\delta(q_1, a) = (q_2, b, L)$$

M 01010010010

0 0ⁱ 0^j 0^k 0^l 0^m || || || w

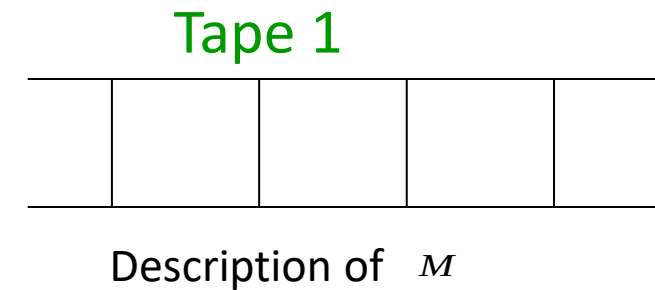
0011 $\xrightarrow{\quad}$ (2) ~~0011~~ enc $\rightarrow$ 013 F
 (3) ~~0011~~ encode
 01 0k

$M \rightarrow A$
 $\rightarrow W$
 $\rightarrow \text{O/P}$
 $U \rightarrow A$
 $\rightarrow R$
 $\rightarrow \text{O/P}$



ORGANIZATION OF UTM

- Input of Universal Turing Machine U:
 - Description of transitions of M
 - Input string of M ie. w



- Describe Turing machine M as a string of symbols
- ie encode M as a string of symbols

ALPHABET ENCODING

Symbols:

a

b

c

d



Encoding:

0

00

000

0000

0 → 0
1 → 00

STATE ENCODING

States:

q1

q2

q3

q4



Encoding:

0

00

000

0000

HEAD MOVE ENCODING

Move:	L	R
		
Encoding:	0	00

TRANSITION ENCODING

Transition:

$$\delta(q_1, a) = (q_2, b, L)$$

Encoding:

0 1 0 1 00 1 00 1 0

separator

TURING MACHINE ENCODING

Transitions:

$$\delta(q_1, a) = (q_2, b, L)$$

$$\delta(q_2, b) = (q_3, c, R)$$

Encoding:

0 1 0 1 00 1 00 1 0 11

separator

0010010001000100

TAPE 1 CONTENTS OF UTM

Tape 1

$e(m)e(w)$
UTM

0011 $\rightarrow M$

010100100

10

(... 11 ... 11 ... 11 ...)

1010

001010010

0 1 0 1 00 1 00 1 0 11

0010010001000100

- A Turing Machine is described with a binary string of 0's and 1's
- **Therefore:** The set of Turing machines forms a language L_u : each string of this language is the binary encoding of a Turing Machine

LANGUAGE OF TURING MACHINES

$L = \{ \text{0101001010...., TM1}$
 $\text{0010010010100...., TM2}$
 $\text{00010100100100...., TM3}$
 $\text{.....} \}$

$\vdash_m, (m \cdot \cdot) (\omega)$

- Define the language L_u as follows:

$L_u = \{ \underline{x} \mid \underline{x} \text{ is in } \{0, 1\}^* \text{ and } \underline{x} = \langle \underline{M}, \underline{w} \rangle \text{ where } M \text{ is a TM encoding and } w \text{ is in } L(M) \}$

THEOREM: UTM EXIST

Proof. Define the *universal language* L_u as the set of all binary strings that encode pairs (M, x) , where M is a Turing machine with binary input alphabet and x is a binary input string so that x lies in $L(M)$. Claim that there is a Turing machine U so that $L(U) = L_u$. Indeed, assume that U has multiple tapes. More precisely, the first tape initially holds the transitions of M , along with the string x . The second tape stores the simulated tape of M , and the third tape holds the state of M . The operations of U can be summarized as follows:

1. Examine the input to check whether the encoding of M is legitimate. If not, U halts without acceptance.
2. Initialize the second tape to contain the input string x in its encoded form (i.e., for each 0 in x place 10 on the tape and for each 1 in x place 100 there).

THEOREM: UTM EXIST

3. Place 0, the start state of M , on the third tape, and move the head of U 's second tape to the first simulated cell.
4. To simulate a transition of M , U searches on its first tape for a string $0^i 1 0^j 1 0^k 1 0^l 1 0^m$ so that 0^i is the state of the third tape, and 0^j is the tape symbol of M that begins at the position of the second tape. If so, U changes the contents of the third tape to 0^k , replaces 0^j on the second tape by 0^l , and keeps the head (N) on the second tape or moves the head on the second tape to the position of the next 1 to the left (L) or to the right (R).
5. If M has no transition that matches the simulated state and tape symbol, then in step 4, no transition will be found. Thus, M halts and U does likewise.
6. If M enters its accepting state, then U accepts (M, x) .

In this way, U simulates M on x so that U accepts the encoded pair (M, x) if and only if M accepts x . This proves the claim. $\square$

TEST YOUR KNOWLEDGE

- Define languages L_0 and L_1 as follows : $L_0 = \{ \langle M, w, 0 \rangle \mid M \text{ halts on } w \}$ $L_1 = \{ \langle M, w, 1 \rangle \mid M \text{ does not halts on } w \}$ Here $\langle M, w, i \rangle$ is a triplet, whose first component. M is an encoding of a Turing Machine, second component, w , is a string, and third component, i , is a bit. Let $L = L_0 \cup L_1$. Which of the following is true ?
 - L is recursively enumerable, but L' is not
 - L' is recursively enumerable, but L is not
 - Both L and L' are recursive
 - Neither L nor L' is recursively enumerable

SUMMARY

- What is undecidability
- Recursive and Recursive enumerable languages

REFERENCE

- Hopcroft J.E., Motwani R. and Ullman J.D, “Introduction to Automata Theory, Languages and Computations”, Second Edition, Pearson Education, 2008